Software Re-Engineering

Lecture 10

Rewrite

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Sometimes a rewrite really is the least worst option. In that case, there are a few things worth taking into consideration if you want your rewrite to proceed smoothly.

- **Deciding the project scope**
- **balanced attitude toward existing code that leads to a successful rewrite**
- **What to do with the DB**

Deciding the project scope

A rewrite usually takes one of three forms.

- **Black-box rewrite**—In a black-box rewrite, the goal is to keep the software's functionality exactly as it is now, but the internals are reimplemented from scratch. This may be to port the software to a new technology stack. In the ideal case, the end user doesn't notice that anything has changed.
- **Brush-up rewrite**—In a brush-up rewrite, an additional goal is to use the rewrite as an opportunity to document, update, and normalize the specification, so that the new software functions differently (hopefully better!) than the old one.
- **Quid pro quo rewrite (Latin: "something for something")**—In a quid pro quo rewrite, the goal is to develop some major new features as part of the rewrite project, in order to convince business stakeholders to green-light it. If it were up to developers, we'd merrily spend all our working time refactoring and rewriting things just for the sake of it, but the people who pay our salaries need to be given some incentive to let us spend weeks or months rebuilding something that already works

Deciding the project scope

Documenting the project scope

Once you've decided what kind of rewrite you're going to do, it's vital that you clearly document this fact, along with any other salient details about the scope of the project

Things to include in project scope document:

New features

Existing features—Are there any features of the existing software that you're planning to remove? Are there any particular features that are must-have or nice to have?

Contd..

Timeliness versus feature-completeness—Are you more interested in releasing something by a certain date or in releasing a product with all the planned features?
Phased releases—Are you planning to make multiple releases, adding more functionality in each release? If so, give a brief summary of the content of each phase.
This is less risky than the alternative, an all-or-nothing Big Bang release at the end of the project, because it gives you the opportunity to get feedback from your users about the new software while you still have time to alter the project's direction. It also highlights any technical issues with the new software at an early stage, when you still have the time to fix them. For an incremental rewrite to work, you'll need to run the old and new software side by side until the new software is complete.

balanced attitude toward existing code that leads to a successful rewrite

The existing code should be respected for a couple of reasons:

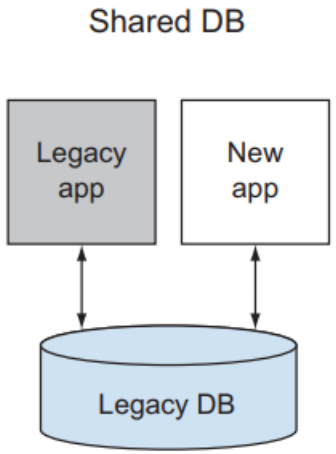
- It contains many years of accumulated bug fixes, performance optimizations, and corner case handling that could be lost in a rewrite if you’re not careful.
- It precisely defines the behavior of the existing software, which can be a useful reference when deciding how the new software should behave

What to do with the DB

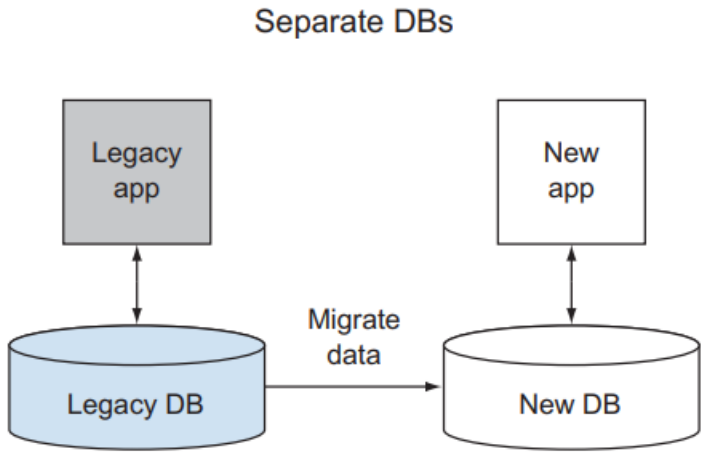
The software you're replacing will most likely have some kind of datastore

Two approaches to replacing legacy software that includes a database: share the existing DB or create a new DB and migrate the data

connect the new software to the existing datastore, so that both the old and new systems are sharing the same DB



create a new datastore and migrate the existing data to it.



Sharing the existing DB

- **Simplicity**—There's no need to worry about migrating data between DBs, or how to recover when those DBs get out of sync, because there's only one DB.
- **No need to update other applications and scripts**—Sometimes the legacy application isn't the only software that interacts with the database. There might be all kinds of scripts, batches, and utilities that connect directly to the database and execute queries against it. If you keep using the existing DB, you won't have to touch any of these scripts.

Sharing the existing DB

- **Can’t choose the datastore technology**—You’re stuck with whatever DB the legacy app happens to be using. If one of the goals of the rewrite is to migrate away from an expensive proprietary DB, then this is a show-stopper.
- **Can’t re-architect**—As part of your rewrite, you might want to split a monolithic application into multiple smaller services, but this is unlikely to be effective unless you also split the DB into a number of isolated datastores.
- **Can’t refactor the schema**—You might want to make major changes to the way data is structured inside the DB, either to make it more consistent with the models in your new application, or simply because the legacy DB hasn’t been maintained and is a horrible mess. But making these changes isn’t easy when the DB is being shared with another application. For every breaking change you want to make to the DB, you’ll have to update the legacy software accordingly.

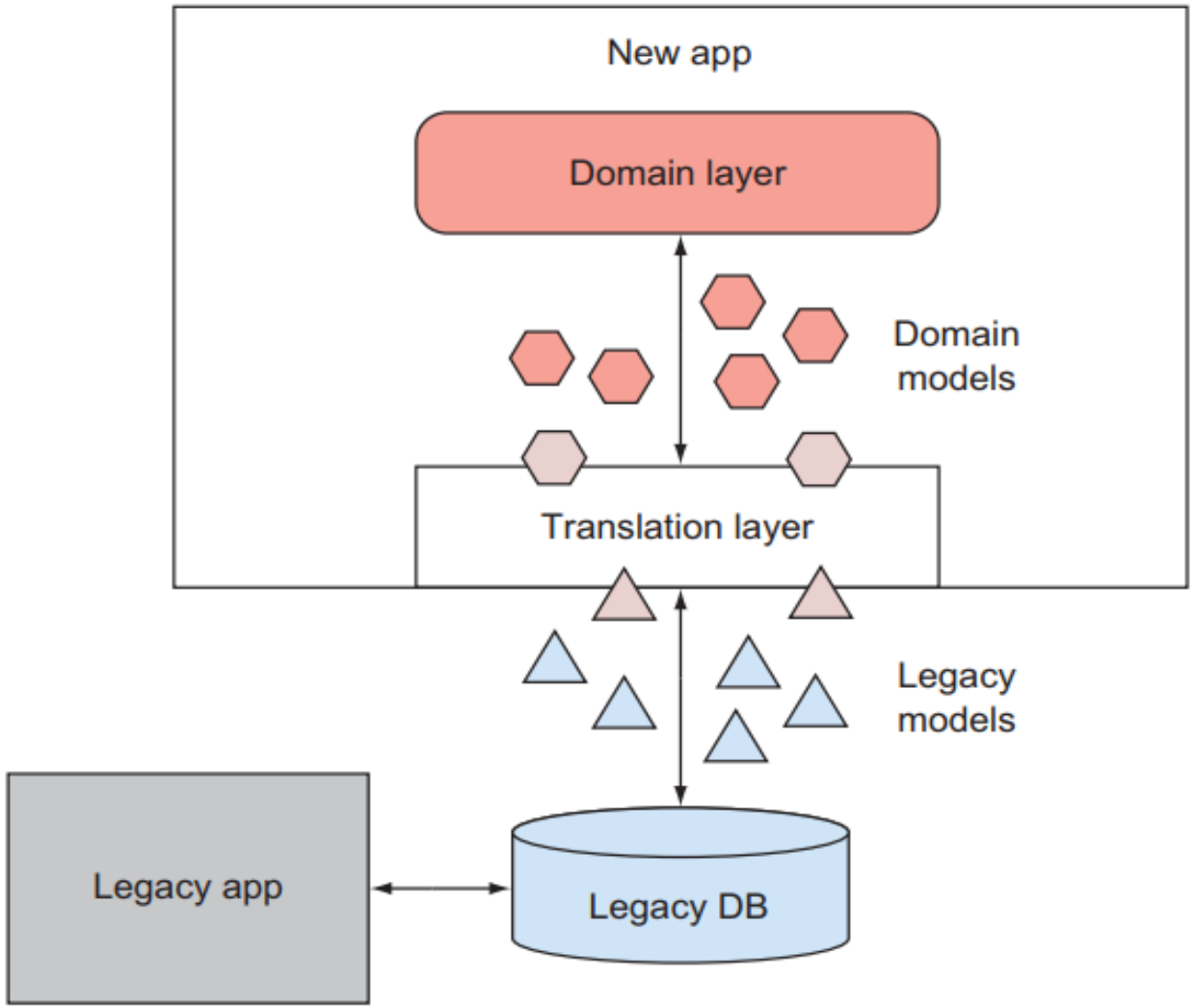
Sharing the existing DB

- **Risk of corrupting data**—Once you release your new software, it will start writing new data to the DB. Ideally this data should be valid in both the old and new applications, so that you have the option of running both applications in parallel for a certain amount of time. This means you can release the new software incrementally, and you can switch to the old application if you find any problems with the new one in production. But if the data written by your new application can’t be read correctly by the old one, this becomes impossible.

Some recommendations...

INVEST HEAVILY IN THE PERSISTENCE LAYER

- if you're using a legacy DB whose schema you can't change, it's easy to end up being constrained by the DB tables and the legacy models that led to their creation.
- You'll likely have a fundamental mismatch between your application's models and the DB in which you have to persist them.
- To handle this mismatch, you'll need to include a translation layer in your application to convert from your **domain-level models** to the **models that are persisted in the DB, and vice versa.**


New app
Domain layer
Domain models
Translation layer
Legacy models
Legacy app
Legacy DB

Example scenario

In the original version of World of RuneQuest, no distinction is made between a Player (a human user) and a Character (that user's persona inside the game). In fact, there is no Character model at all, only Player. But in the rewrite, we want Players to be able to create multiple Characters and choose between them whenever they play.

In the legacy DB design, each Player is represented by a record in the player table. This record contains information about the human user (their username, hashed password, and so on) and about that user's persona in the game (such as species, strength, and magic points).

One way would be to leave each Player's first Character in the player table, while putting all the Player's other Characters in a new character table.

If a Player creates five Characters, one will be stored in the player table and the other four will be in the character table.

In terms of compatibility with the legacy application, this is a good solution. The legacy application will ignore the newly created table, and as far as it's concerned, nothing has changed.

But in the context of our new application, this is a horrible hack. In our domain models, we don't want to know anything about the arbitrary split between first character and other four

This is where the translation layer comes to the rescue. We can hide all the gory details of translating between DB models and domain models in the translation layer, so that hacks performed for the sake of legacy compatibility don't infect the core of our application

A translation layer isn't the only possible solution. You might prefer to do the translation at the database level, using views, rather than in code.

You could do the following:

1. Create the new character table and move the character-related fields out of player into character.
2. Create a view called player_character that contains the data the legacy app expects to find in the player table.
3. Make a small change to the legacy app so that it queries the new player _character view instead of the player table.
4. Make the view writable, if possible, so that the correct records are updated when the legacy app runs an SQL UPDATE against the view.

It is usually preferred keeping logic like this out of the DB and implementing it in code at the application layer, where it's easier to read, easier to test, and more visible to developers.

When parts of a legacy application are implemented in DB triggers, stored procedures, or anywhere else that can't be found just by looking through the source code, it can be very confusing for developers who come along to maintain the application later.

PLAN FOR THE DAY YOU GAIN CONTROL OF THE DB

- As long as the legacy application is still running, you'll be restricted in the changes you can make to the DB schema. You can safely add new tables, as the legacy application will just ignore them, and you can probably add indexes to existing tables to improve performance, but that's about it.
- If there's anything you don't like about the current DB structure, you'll have to put up with it and work around it. But as soon as you switch off the legacy system, you can take full control of the database. At this point you're free to alter the DB however you like and refactor all the parts that don't fit your new domain models or are simply a mess.
- Because it might be months, or even years, before that day comes, **it's a good idea to keep a running list of all the DB changes you want to make, just so you don't forget them**. This could be a simple text file, stored in version control along with the project's source code, full of comments like "remove the game.created_by column because it's not used anymore"

PLAN FOR THE DAY YOU GAIN CONTROL OF THE DB

For each change you want to make, you should leave some kind of record near any source code that’s currently working around the problem, so you can find and update this source code once you’ve refactored the DB.

Creating a new DB

REAL-TIME SYNCHRONIZATION Depending on your application's needs, you may need all writes in one DB to also appear instantly in the other, implying some kind of real-time notifications between the applications. This might be implemented using database triggers

Different approaches for synchronizing DB writes in real time

- In the **first case**, a DB trigger is used to copy writes directly from one DB to the other.
- In the **second case**, a DB trigger writes the update to a queue, which is then consumed by the application. The reason for using a queue, rather than sending the data directly to an API endpoint on the application, is to avoid losing writes when the application is not running.
- In the **third case**, the application writes to the queue whenever it updates data in the DB. The problem with this approach is that it requires implementation work on the legacy application side. You'll need to add code to the legacy application to send a message to a queue every time it writes to the DB,

Creating a new DB

BATCH SYNCHRONIZATION

- copy a batch of existing updates from one datastore to the other.
- The tool should support both
  - a full replication of the whole datastore (it is useful when a bug in your new application corrupts its own datastore beyond repair, which will happen at least once during development)

  and
  - replication of only updates that match a certain query, such as all updates that occurred in the last hour (it is useful for recovering quickly when some real-time updates are missed, which will also happen occasionally).

MONITORING

- You’ll need monitoring tools to continuously check that the data replication is working as expected, ensuring both datastores contain the same data.
- When the monitoring system discovers that any updates have been missed, it should at least send an alert to the development team, and if possible recover automatically by triggering a re-replication of the affected data.

REPLICATING TRAFFIC

- copying all incoming requests from clients and sending them to both applications (but, of course, returning only one application's responses
- Each application will receive and process all requests and update its own database, caches, and so on, blissfully unaware of the other's existence. However, this approach is only applicable under certain conditions:
  - The applications don't share a DB (otherwise they would both try to write the same data to it at the same time).
  - The new application is relatively stable (it's running and able to receive traffic most of the time, and you have a way to recover any data that it missed out on whenever it wasn't running)

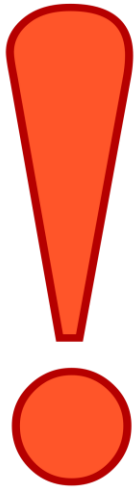
Inter-app communication

- When you develop an application, you usually assume that it will be the sole writer of its own database.
- When we run a legacy application and its replacement in parallel, they are no longer the sole writers of their respective DBs, so the assumption becomes invalid.
- If they’re sharing the DB, then obviously they are both writing to it, so there are two writers. And even if they have separate DBs, the other application (or a batch replication tool) might be replicating writes directly to the DB.
- The essence of the problem is that the ***applications have no way of knowing when somebody else has made a write to their DB***.
- But if we notify the application whenever we make a write, it can take action accordingly. For example, if it receives a notification saying “I just updated User 123 in your DB,” then it can remove all data about User 123 from its in-memory caches.

Inter-app communication

To support these notifications, we need to implement two things in both the existing legacy application and the new application

- A notifier that sends a notification to the other application every time it writes to the DB
- An endpoint that can receive notifications and act upon them



- On the legacy application side, it doesn’t really matter how you implement this. As long as it works, the implementation can be as dirty as you like.
- But in the new application, the notification system should be implemented with an eye to the future, bearing in mind that it will become obsolete as soon as the legacy application is shut down.
- In other words, it should be implemented in a way that’s easy to remove.

One design pattern that makes sense in this case is **Event Bus (also known as Publish/Subscribe)**.

Whenever the persistence layer writes data to the DB, it pushes an event onto the bus, which is then broadcast to all listeners.

A listener then reacts to the event by sending a corresponding notification to the legacy application.

When the legacy application is shut down, all we need to do is remove the listener.

We can leave the Event Bus system in place, as it may prove useful for other features in the future.

Mandatory readings

Chapter 6, Chris Birchall - Re-Engineering Legacy Software- Manning Publications (2016)